

Chapter 3

Advanced Data Wrangling and Transformation

Pandas Merging

Merging is used to combine two DataFrames based on one or more common columns (keys).

Types of Merge

Type	Description
inner	Only matching keys
left	All rows from left DF
right	All rows from right DF
outer	All rows from both

Pandas Merging

```
import pandas as pd

df1 = pd.DataFrame({
    'id': [1, 2, 3],
    'name': ['Ram', 'Shyam', 'Hari']
})

df2 = pd.DataFrame({
    'id': [1, 2, 4],
    'marks': [80, 75, 90]
})

merged = pd.merge(df1, df2, on='id', how='inner')

print(merged)
```

	id	name	marks
0	1	Ram	80
1	2	Shyam	75

Pandas Joining

In **Pandas**, `join()` is used to **combine two DataFrames using their index** (by default).

Simpler and faster than `merge()` when data is indexed

Key Characteristics of `join()`

- Joins on **index** by default
- Supports join types: `left`, `right`, `inner`, `outer`
- Best when both DataFrames share a common index

Pandas Joining

```
import pandas as pd

df1 = pd.DataFrame({
    'Name': ['Ram', 'Shyam', 'Hari']
}, index=[1, 2, 3])

df2 = pd.DataFrame({
    'Marks': [80, 75, 90]
}, index=[1, 2, 4])

result = df1.join(df2)

print(result)
```

Pandas reshaping

Reshaping means changing the structure (shape) of data without changing the actual values.

Common Reshaping Functions

Function	Purpose
<code>stack()</code>	Columns → rows
<code>unstack()</code>	Rows → columns
<code>melt()</code>	Wide → long
<code>pivot()</code>	Long → wide
<code>pivot_table()</code>	Pivot with aggregation

Pandas reshaping

```
import pandas as pd

df = pd.DataFrame({
    'Math': [80, 85],
    'Science': [90, 88]
}, index=['Ram', 'Shyam'])

print(df)
```

```
stacked = df.stack()
print(stacked)
```

	Math	Science
Ram	80	90
Shyam	85	88

Ram	Math	80
	Science	90
Shyam	Math	85
	Science	88

Pandas Pivoting

Pivoting is a reshaping operation used to transform data from long format to wide format.

It reorganizes data by:

- Selecting an index
- Creating new columns
- Filling values from another column

Pivoting Functions in Pandas

Function	Use
<code>pivot()</code>	Simple pivot (no duplicates)
<code>pivot_table()</code>	Pivot with aggregation (handles duplicates)

Pandas Pivoting

```
import pandas as pd

df = pd.DataFrame({
    'Student': ['Ram', 'Ram', 'Shyam', 'Shyam'],
    'Subject': ['Math', 'Science', 'Math', 'Science'],
    'Marks': [80, 90, 85, 88]
})

print(df)
```

```
pivot_df = df.pivot(
    index='Student',
    columns='Subject',
    values='Marks'
)

print(pivot_df)
```

Pandas Pivoting

✓ Output

javascript

Subject	Math	Science
Student		
Ram	80	90
Shyam	85	88

🧠 Explanation

- Student → row index
- Subject → column headers
- Marks → values

Handling Missing Values

Missing data refers to values that are absent, undefined, or not recorded in a dataset.

In Pandas, missing values are commonly represented as:

- NaN – Not a Number (numeric data)
- None – Python null object
- NaT – Not a Time (datetime data)

Handling missing data is essential to ensure accurate analysis, error-free computation, and reliable machine learning models.

Handling Missing Values

Function	Purpose
<code>isnull()</code> / <code>isna()</code>	Detect missing values
<code>notnull()</code>	Detect non-missing values
<code>dropna()</code>	Remove missing values
<code>fillna()</code>	Fill missing values
<code>replace()</code>	Replace custom missing values
<code>interpolate()</code>	Estimate missing values

Detecting Missing Data

```
import pandas as pd
import numpy as np

df = pd.DataFrame({
    'Name': ['Ram', 'Shyam', None],
    'Marks': [80, np.nan, 90]
})

print(df.isnull())
```

Fill with Mean (Numerical Column)

```
df['Marks'].fillna(df['Marks'].mean(), inplace=True)
```

Counting Missing Values

```
df.isnull().sum()
```

Removing Missing Data

```
df.dropna()
```

Filling Missing Data (fillna())

```
df.fillna(0)
```

Handling categorical data

Categorical data represents values that belong to **fixed categories or labels** rather than numerical values.

Examples

- Gender: Male, Female
- Grade: A, B, C
- City: Kathmandu, Pokhara, Biratnagar

In Pandas, categorical data is usually stored as:

- `object` dtype (strings)
- `category` dtype (optimized categorical type)

Handling categorical data is essential for **data analysis**, **visualization**, and **machine learning**.

Handling categorical data

Function	Purpose
<code>astype('category')</code>	Convert to categorical type
<code>value_counts()</code>	Frequency of categories
<code>unique()</code>	Get unique categories
<code>nunique()</code>	Count unique categories
<code>get_dummies()</code>	One-hot encoding
<code>map()</code>	Map categories to numbers
<code>replace()</code>	Replace category values
<code>cat.codes</code>	Convert to integer codes

Handling categorical data

```
import pandas as pd
```

```
df = pd.DataFrame({  
    'Name': ['Ram', 'Shyam', 'Hari'],  
    'Gender': ['Male', 'Male', 'Female'],  
    'City': ['Kathmandu', 'Pokhara', 'Kathmandu']  
})
```

```
df.dtypes
```

Unique Categories

python

```
df['Gender'].unique()  
df['City'].nunique()
```



Frequency Count

python

```
df['City'].value_counts()
```

Renaming / Replacing Categories

```
df['Gender'] = df['Gender'].replace({  
    'Male': 'M',  
    'Female': 'F'  
})
```


Handling categorical data

Mapping Categories to Numbers

```
df['Gender_Code'] = df['Gender'].map({  
    'M': 1,  
    'F': 0  
})
```

One-Hot Encoding (`get_dummies()`)

```
encoded_df = pd.get_dummies(df, columns=['City'])  
print(encoded_df)
```

Handling Time-Series Data

Time series data is data collected over time, where each record is associated with a date or time.

Examples:

- Daily sales
- Monthly temperature
- Hourly sensor data

In Pandas, time series handling means:

1. Converting dates into datetime format
2. Using dates as index
3. Performing time-based operations (filtering, resampling, shifting)

Handling Time-Series Data

Function	Purpose
<code>pd.to_datetime()</code>	Convert to datetime
<code>set_index()</code>	Set date as index
<code>date_range()</code>	Create date sequence
<code>.dt</code>	Extract year, month, day
<code>resample()</code>	Change frequency
<code>shift()</code>	Lag / lead values
<code>rolling()</code>	Moving calculations
<code>asfreq()</code>	Set fixed frequency

Handling Time-Series Data

```
import pandas as pd

data = {
    'Date': ['2024-01-01', '2024-01-02', '2024-01-03',
            '2024-01-04', '2024-01-05'],
    'Sales': [100, 120, 90, 130, 110]
}

df = pd.DataFrame(data)
print(df)
```

	Date	Sales
0	2024-01-01	100
1	2024-01-02	120
2	2024-01-03	90
3	2024-01-04	130
4	2024-01-05	110

Handling Time-Series Data

1 Convert Date Column to Datetime

Theory

Pandas must know that the column represents **dates**, not strings.

Code

```
python
```

```
df['Date'] = pd.to_datetime(df['Date'])  
print(df.dtypes)
```

2 Set Date as Index (VERY IMPORTANT)

Theory

Time-series operations work on **DatetimeIndex**.

Code

```
python
```

```
df.set_index('Date', inplace=True)  
print(df)
```

Handling Time-Series Data

3 Date-Based Selection (Filtering)

Theory

You can select data using **date ranges**.

Code

```
python
```

```
df['2024-01-02':'2024-01-04']
```

4 Extract Date Components

Theory

Extract parts of the date for analysis.

Code

```
python
```

```
df['Year'] = df.index.year  
df['Month'] = df.index.month  
df['Day'] = df.index.day
```

```
print(df)
```

Handling Time-Series Data

5 Resampling (Change Time Frequency)

Theory

Resampling means **changing time intervals**
(e.g., daily → monthly)

Code

```
python

monthly_sales = df.resample('M').sum()
print(monthly_sales)
```

Common Frequencies:

- `D` → Daily
- `M` → Monthly
- `Y` → Yearly

6 Shift (Lag / Lead Data)

Theory

Used to compare current values with previous values.

Code

```
python

df['Previous_Day_Sales'] = df['Sales'].shift(1)
print(df)
```

Handling Time-Series Data

Rolling Window (Moving Average)

Theory

Used for smoothing data and trend analysis.

Code

```
python
```

```
df['3_Day_Avg'] = df['Sales'].rolling(window=3).mean()  
print(df)
```


Handling Time-Series Data

Create Date Range

Theory

Generate time-based index automatically.

Code

```
python
```

```
dates = pd.date_range(start='2024-01-01', periods=7, freq='D')  
print(dates)
```

Handling Time-Series Data

Handling Missing Dates

Theory

Ensure continuous time intervals.

Code

python

```
df = df.asfreq('D')  
df.fillna(method='ffill', inplace=True)
```

Feature transformation, scaling and encoding

Definition: Changing or creating features to make data more suitable for analysis or ML.

It can be **mathematical, logical, or derived from existing features.**

Examples:

- Log, square root, or power transformations
- Creating new features from existing ones (like $\text{total_price} = \text{quantity} \times \text{price}$)
- Extracting date parts (year, month, day)

Feature transformation

```
import pandas as pd
import numpy as np

df = pd.DataFrame({
    'income': [2000, 5000, 10000, 0],
    'quantity': [5, 3, 8, 2],
    'price': [100, 200, 150, 300],
    'date': pd.to_datetime(['2023-01-01', '2023-02-15', '2023-03-10', '2023-04-01'])
})

# Log Transformation
df['log_income'] = np.log1p(df['income'])

# Feature Creation
df['total_price'] = df['quantity'] * df['price']

# Date Extraction
df['year'] = df['date'].dt.year
df['month'] = df['date'].dt.month

print(df)
```

Feature Scaling

Definition: Adjusting numeric values to a **common scale** so models aren't biased by large values.

Important for models like **KNN, SVM, PCA, Neural Networks**.

Min-Max Scaling (0–1 range):

$$\text{df['income_scaled']} = (\text{df['income']} - \text{df['income'].min()}) / (\text{df['income'].max()} - \text{df['income'].min()})$$

Z-Score Standardization

$$\text{df['income_z']} = (\text{df['income']} - \text{df['income'].mean()}) / \text{df['income'].std()}$$

Feature Encoding

Converting **categorical (text) data into numbers** so ML models can use them.

a) Label Encoding (Assign numbers to categories)

python

```
df = pd.DataFrame({'gender': ['Male', 'Female', 'Female', 'Male']})  
df['gender_encoded'] = df['gender'].map({'Male': 0, 'Female': 1})
```

b) One-Hot Encoding (Create dummy variables)

python

```
df = pd.DataFrame({'city': ['London', 'Paris', 'London', 'Tokyo']})  
df_encoded = pd.get_dummies(df, columns=['city'], drop_first=True)
```

Memory Optimization and efficient data processing

In data science, large volumes of data are collected, stored, processed, and analyzed to extract insights and build predictive models. These datasets often contain millions of records and numerous features. Since computer memory (RAM) is limited, inefficient use of memory can cause slow execution, system crashes, or failure of data processing tasks.

Therefore, memory optimization and efficient data processing are essential to ensure scalable, fast, and reliable data science workflows.

Memory Optimization and efficient data processing

What is memory optimization?

Memory optimization refers to the practice of reducing the amount of RAM required to store and process data without losing its meaning or usefulness.

The main goal is:

- To represent data using the **minimum necessary memory**
- While preserving data accuracy and structure

Memory optimization does **not** mean deleting important data; it means storing the same information more efficiently.

Why memory optimization is necessary in data science

Memory optimization is important because:

1. **Large datasets**
Modern data science works with big data from sensors, social media, transactions, and logs.
2. **Limited hardware resources**
Not all systems have high RAM capacity, especially personal computers or cloud instances with budget limits.
3. **Performance improvement**
Lower memory usage leads to faster data access and processing.
4. **Scalability**
Efficient memory usage allows systems to handle growing datasets.
5. **Cost efficiency**
In cloud environments, memory usage directly impacts cost.

Understanding memory usage in data processing

Every dataset occupies memory based on:

- Number of records
- Number of features
- Type of data stored in each feature

Some data representations consume more memory than necessary. For example:

- Storing small numerical values in large storage formats
- Storing repeated text values multiple times

These lead to memory waste.

Data representation and its impact on memory

Numerical data

Numerical data can be stored using different sizes. If large storage formats are used for small values, memory is wasted. Choosing the smallest possible representation that can still store all values reduces memory consumption.

Categorical data

Categorical data contains repeated values such as gender, city, or status. Storing the same text repeatedly increases memory usage. A more efficient approach is to store unique values once and reference them, which significantly reduces memory consumption.

Redundant and irrelevant data

Many datasets contain:

- Unused columns
- Duplicate records
- Temporary or intermediate features

These unnecessarily occupy memory and slow down processing.

Effective data loading

Efficient data processing begins **before** data analysis starts.

What can be done:

- Load only the data that is required for analysis
- Avoid loading unused features
- Avoid loading the entire dataset if only a portion is needed

This prevents unnecessary memory usage from the start.

Efficient data processing

Efficient data processing means performing operations in a way that:

- Uses minimum memory
- Avoids unnecessary duplication of data
- Completes tasks faster

Avoiding unnecessary data duplication

Many data processing operations create copies of data. Each copy requires additional memory.

What can be done:

- Modify data carefully instead of repeatedly creating new versions
- Remove temporary or intermediate datasets after use
- Design workflows that minimize repeated transformations

Processing data in parts (chunk-based processing)

When a dataset is too large to fit into memory:

- The data should be processed in smaller parts
- Each part is processed independently
- Results are combined at the end

This allows very large datasets to be handled using limited memory.

Efficient data storage formats

The format in which data is stored affects memory and processing speed.

Efficient storage formats:

- Reduce file size
- Allow faster reading and writing
- Store metadata for better data handling

Using efficient formats improves both memory usage and processing efficiency.

Sampling and data reduction

During exploration and experimentation, working on the full dataset is often unnecessary.

What can be done:

- Use a representative subset of data
- Reduce feature dimensionality
- Remove noise and irrelevant attributes

This reduces memory usage and speeds up analysis.

Relationship between memory and performance

Memory usage and processing speed are closely related:

- Lower memory usage improves CPU cache efficiency
- Reduces system overhead
- Minimizes delays caused by memory swapping

As a result, optimized memory usage leads to faster data processing.

Importance in machine learning workflows

Memory optimization is crucial in machine learning because:

- Feature engineering can multiply data size
- Model training requires repeated data access
- Inefficient memory usage can limit model complexity

Efficient data processing enables smoother and more reliable model development.

Building a Reusable Data Cleaning Pipeline

In data science, raw data is rarely clean or ready for analysis. It often contains missing values, duplicates, inconsistencies, outliers, and incorrect formats. Data cleaning is therefore a critical step before data analysis and machine learning.

However, performing data cleaning manually for every dataset is:

- Time-consuming
Error-prone
- Difficult to reproduce

To solve this, data scientists build **reusable data cleaning pipelines**.

What is a Data Cleaning Pipeline?

A **data cleaning pipeline** is a structured and sequential process where raw data passes through multiple predefined steps to transform it into clean, consistent, and analysis-ready data.

A **reusable pipeline** is designed so that:

- The same cleaning logic can be applied to different datasets
- Steps can be repeated consistently
- The process is automated and reproducible

Why Reusable Pipelines are Needed

Ensure consistency

The same cleaning rules are applied every time.

Save time and effort

No need to rewrite cleaning logic for each dataset.

Improve reproducibility

Results can be recreated easily.

Reduce human errors

Automated steps avoid manual mistakes.

Support scalability

Large datasets and repeated workflows become manageable.

Common Data Quality Issues Addressed by Pipelines

Missing values

Duplicate records

Incorrect data types

Inconsistent categorical values

Outliers and noisy data

Invalid or corrupted entries

Feature scaling and encoding

Components of Data Cleaning Pipeline (Data Ingestion)

Data ingestion is the process of loading raw data from external sources such as files, databases, or APIs into the system. A reusable pipeline must support flexible and consistent data loading.

```
import pandas as pd

def load_data(file_path):
    df = pd.read_csv(file_path)
    return df
```


Components of Data Cleaning Pipeline (Data Validation)

Data validation ensures that the dataset has the required structure before cleaning begins. It checks for missing columns, incorrect schema, or invalid formats to prevent errors later in the pipeline.

```
def validate_data(df, required_columns):  
    for col in required_columns:  
        if col not in df.columns:  
            raise ValueError(f"Missing required column: {col}")  
    return df
```

Components of Data Cleaning Pipeline (Handling Missing Values)

Missing values reduce data quality and can negatively affect analysis and machine learning models. A pipeline must apply consistent rules to handle missing data.

Common strategies:

- Replace numerical values with mean/median
- Replace categorical values with default labels
- Remove records if necessary

Components of Data Cleaning Pipeline (Handling Missing Values)

```
def handle_missing_values(df):  
    for col in df.select_dtypes(include="number").columns:  
        df[col].fillna(df[col].median(), inplace=True)  
  
    for col in df.select_dtypes(include="object").columns:  
        df[col].fillna("Unknown", inplace=True)  
  
    return df
```

Components of Data Cleaning Pipeline (Removing Duplicate Records)

Duplicate records increase dataset size and may bias statistical analysis and machine learning models. Removing duplicates improves data accuracy and memory efficiency.

```
def remove_duplicates(df):  
    df.drop_duplicates(inplace=True)  
    return df
```

Components of Data Cleaning Pipeline (Data Type Standardization)

Data often arrives in incorrect or inconsistent formats. Standardizing data types ensures correct calculations, comparisons, and memory efficiency.

```
def standardize_data_types(df):  
    if "age" in df.columns:  
        df["age"] = df["age"].astype(int)  
  
    if "salary" in df.columns:  
        df["salary"] = df["salary"].astype(float)  
  
    return df
```

Components of Data Cleaning Pipeline (Cleaning Categorical Data)

Categorical data may contain inconsistent values due to spelling differences, capitalization, or abbreviations. Standardizing these values improves data quality and model performance.

```
def clean_categorical_data(df):  
    if "gender" in df.columns:  
        df["gender"] = df["gender"].str.lower().str.strip()  
        df["gender"] = df["gender"].replace({  
            "m": "male",  
            "f": "female"  
        })  
    return df
```

Components of Data Cleaning Pipeline (Handling Outliers)

Outliers are extreme values that may arise due to errors or rare events. They can distort statistical analysis and machine learning models. Pipelines handle outliers using predefined rules.

```
def handle_outliers(df, column):  
    q1 = df[column].quantile(0.25)  
    q3 = df[column].quantile(0.75)  
    iqr = q3 - q1  
  
    lower = q1 - 1.5 * iqr  
    upper = q3 + 1.5 * iqr  
  
    df = df[(df[column] >= lower) & (df[column] <= upper)]  
    return df
```

Components of Data Cleaning Pipeline (Feature Selection)

Feature selection removes unnecessary or irrelevant columns, reducing noise, memory usage, and computational cost.

```
def select_features(df, selected_columns):  
    return df[selected_columns]
```


Components of Data Cleaning Pipeline (Feature Transformation)

Feature transformation creates new features or modifies existing ones to improve model learning and data representation.

```
def create_features(df):  
    if "salary" in df.columns:  
        df["salary_log"] = df["salary"].apply(lambda x: x if x > 0 else 0)  
    return df
```

Components of Data Cleaning Pipeline

```
def data_cleaning_pipeline(file_path):  
    df = load_data(file_path)  
    df = validate_data(df, ["age", "salary", "gender"])  
    df = handle_missing_values(df)  
    df = remove_duplicates(df)  
    df = standardize_data_types(df)  
    df = clean_categorical_data(df)  
    df = handle_outliers(df, "salary")  
    df = create_features(df)  
    df = select_features(df, ["age", "salary", "salary_log", "gender"])  
    return df
```